OPEN SOURCE ENGINEERING

Name : V Sri Anjali

Student ID : 2400031598

Semester : ODD

Academic Year : 2025–2026

Course Code : 24CS02EF

Contents

1 About the Linux Distribution I Used

1.1 Introduction

For this project, I used the Linux distribution called **Ubuntu**. I selected Ubuntu because it is easy to install, beginner friendly, and widely used in the open-source community. It gives a simple desktop, good hardware support and regular security updates.

1.2 Desktop Experience

Ubuntu uses the **GNOME** desktop environment. The interface is very simple:

- A left-side dock for favourite applications.
- Activities view to see all open windows.
- Search box to quickly open apps and files.

Because of this, it feels comfortable even for someone coming from Windows.

1.3 Software Management

Ubuntu uses the apt package manager. To install software, we can run:

```
sudo apt update
sudo apt install packagename
```

There is also a GUI "App Store" and support for **Snap** packages, so it is very easy to add or remove applications.

2 Encryption and GPG

2.1 Why Encryption is Needed

Encryption protects our data from others. Even if someone gets our files or laptop, they cannot read the content without the correct key or password.

In Linux we mainly use:

- **Full Disk Encryption** – entire disk is encrypted.
- **File or Folder Encryption** – only some files are encrypted.

2.2 What is GPG?

GPG (GNU Privacy Guard) is a popular tool for encryption and digital signatures.

It works with two keys:

- **Public key** – we can share with others.
- **Private key** – we must keep secret and protect with a passphrase.

If someone encrypts a file with my public key, only my private key can open it.

2.3 Basic GPG Commands I Used

Generate a key pair:

```
gpg --full-generate-key
```

Encrypt a file with a password (symmetric encryption):

```
gpg -c notes.txt
```

Encrypt a file for a specific person:

```
gpg --encrypt --recipient "friend@example.com" secret.txt
```

Decrypt an encrypted file:

```
gpg --decrypt secret.txt.gpg
```

3 Sending Encrypted Email

3.1 Preparation

To send encrypted email using GPG, both sender and receiver must:

1. Generate their own GPG key pair.
2. Export and exchange their **public keys**.
3. Import the other person's public key into their keyring.

3.2 Using Thunderbird

I used the **Thunderbird** email client.
Steps:

- Added my email account in Thunderbird.
- Enabled OpenPGP support.
- While composing a new email, I selected options to **Encrypt** and **Sign**.

Thunderbird automatically used my private key to sign the message and the receiver's public key to encrypt it.

3.3 Receiver Side

On the receiver side:

- Thunderbird uses the receiver's private key to decrypt the message.
- It uses my public key to verify the digital signature.

If both steps succeed, the receiver knows that the message really came from me and was not changed.

4 Five Privacy Tools from PRISM Break

4.1 Tor Browser

Tor Browser routes internet traffic through several relay nodes. This hides my IP address and location. It is useful when strong anonymity is needed.

4.2 Firefox

Mozilla Firefox is an open-source web browser. It supports Enhanced Tracking Protection and many privacy add-ons like uBlock Origin. With correct settings it gives good privacy for daily browsing.

4.3 KeePassXC

KeePassXC is a password manager. All passwords are stored in a single encrypted database file on my system. Only one master password is needed. Nothing is uploaded to any cloud server.

4.4 Thunderbird

Thunderbird is an open-source email client. It supports GPG/PGP encryption and digital signatures. This helps me send and receive secure emails from my laptop.

4.5 Signal

Signal is a secure messaging application. It uses end-to-end encryption for messages, calls and media, so only the sender and receiver can read the content.

5 Open Source License I Used

5.1 MIT License

For my project work I chose the **MIT License**. It is a very short and permissive open-source license.

5.2 Permissions

Under the MIT License, anyone can:

- Use the software for any purpose.
- Copy and modify the code.
- Distribute it.
- Even sell it as part of a commercial product.

5.3 Conditions

The conditions are simple:

- Keep the original copyright notice.
- Keep the MIT license text in the project.

5.4 No Warranty

The license clearly says the software is provided “as is”. That means the author is not responsible for any damage or problems caused by using the software.

6 Self-Hosted Server: Jellyfin

6.1 About Jellyfin

For the self-hosting part of the course, I chose **Jellyfin**. Jellyfin is a free and open-source media server. It lets me organise, stream and share my movies, TV shows and music from my own machine. All data stays in my control; nothing is sent to third-party cloud services.

6.2 Why Jellyfin is Useful

- I can access my media library from phone, laptop or smart TV on the same network.
- It supports multiple users with different profiles.
- It is fully open-source with no subscription or hidden tracking.

6.3 Installation Steps on Ubuntu

I installed Jellyfin on Ubuntu with the following commands:

```
sudo apt install apt-transport-https
sudo add-apt-repository universe
sudo apt update
sudo apt install jellyfin
```

After installation, I opened Jellyfin in my browser:

```
http://localhost:8096
```

From there I created an admin account and added my media folders. Now my local Jellyfin server is ready.

6.4 Localized Document and Poster

I prepared a small guide in simple English and also explained Jellyfin features to my friends in Telugu. I also designed a poster for Jellyfin. It highlights:

- Streaming of movies, TV shows and music.
- Open-source and subscription-free nature.
- Support for multiple users and remote media access.

A sample of my Jellyfin poster is shown in Figure 1.

7 My Open Source Contributions (Pull Requests)

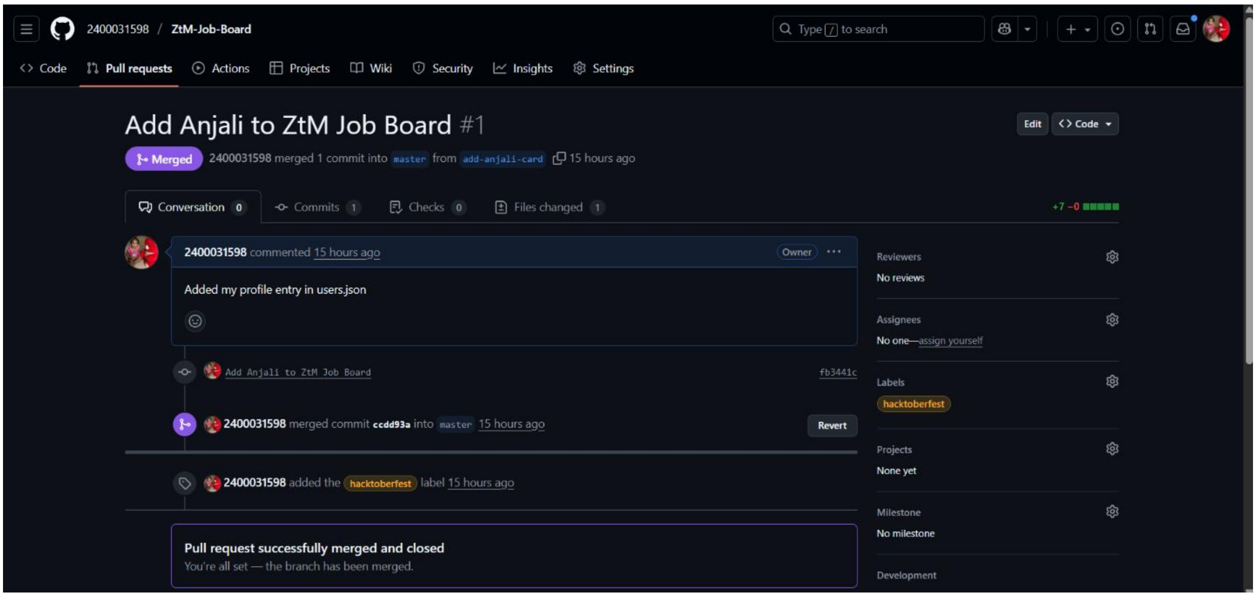
During this course I made several Pull Requests (PRs) on GitHub. Below I describe each contribution in simple words.

7.1 PR 1: Add Anjali to ZtM Job Board

Repository: 2400031598/ZtM-Job-Board

Issue and Goal

The project maintains a job board with student profiles. The task was to add my own profile entry so that my details appear on the board.



What I Did

- Forked the repository and cloned it.
- Edited users.json and added my profile entry.
- Committed the change and opened a PR named *"Add Anjali to ZtM Job Board"*.

The PR was successfully **merged**. A screenshot is shown in Figure 2.

7.2 PR 2: Add anjali.txt File

Repository: 2400031598/my-merge-test

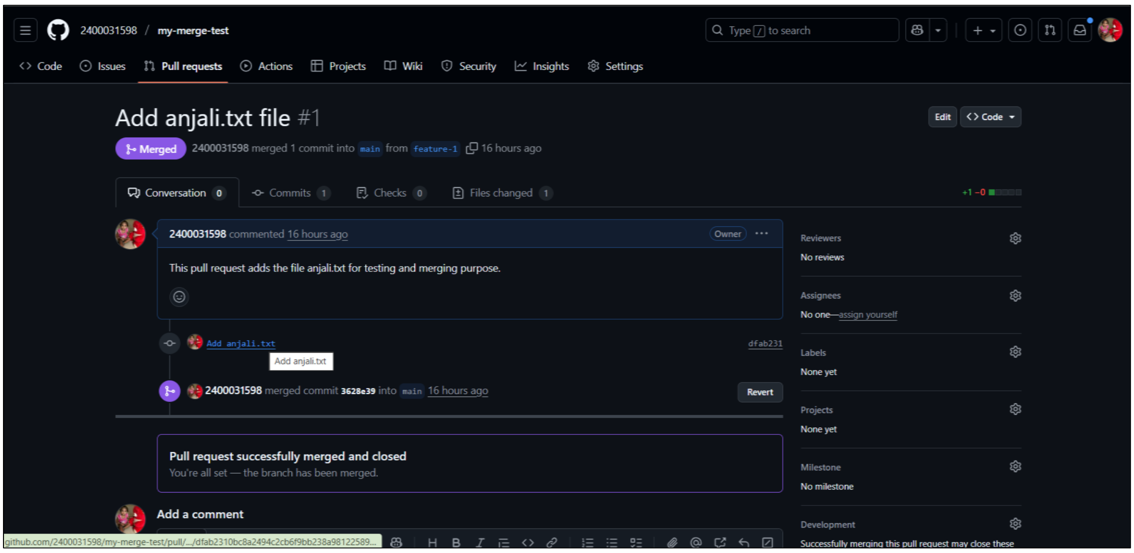


Figure 2: PR: Add Anjali to ZtM Job Board (merged)

Issue and Goal

This repository was used to practice the Git branching and merging workflow. The goal was to add a simple text file through a feature branch and merge it through a PR.

What I Did

- Created a new branch called feature-1.
- Added a new file anjali.txt with a short message.
- Committed the file and opened a PR titled *"Add anjali.txt file"*.

The PR was **merged** into the main branch. Screenshot is shown in Figure 3.

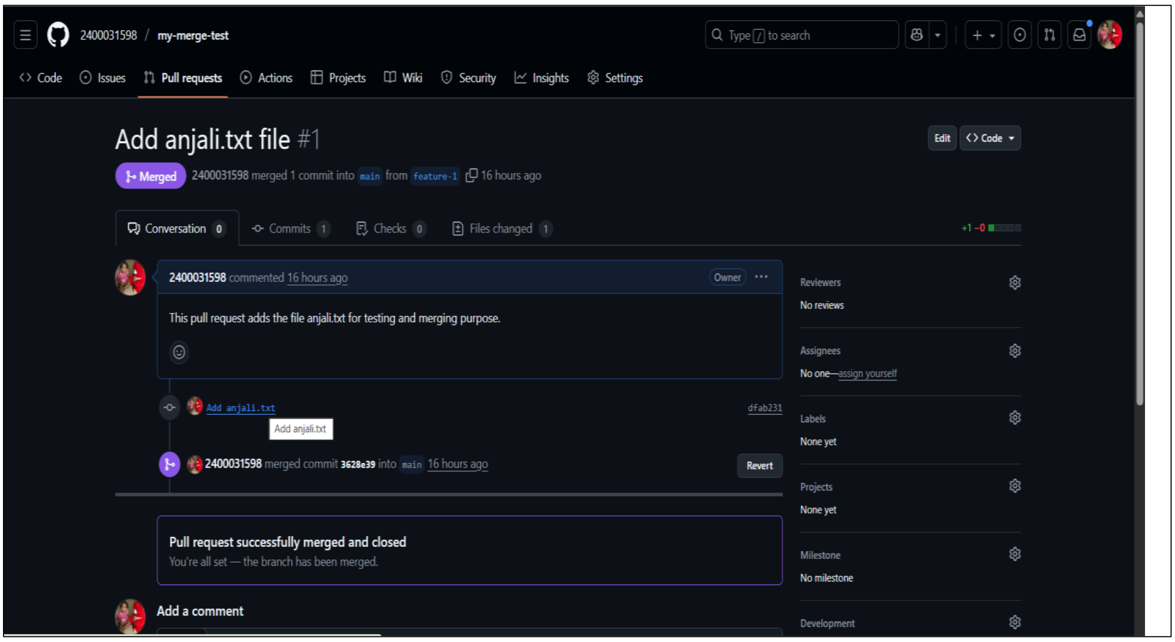


Figure 3: PR: Add anjali.txt file (merged)

7.3 PR 3: Hacktoberfest Java Program

Repository: AdarshAddee/Hacktoberfest2022_for_Beginers

Issue and Goal

The repository accepts beginner-friendly code contributions. I decided to add a Java program to calculate the Greatest Common Divisor (GCD) of the sum of odd and even numbers.

What I Did

- Wrote the file gcdOfOddEvenSums.java.

- The program reads a list or range of numbers, separates odd and even sums, and finds the GCD.
- Opened a PR titled *"Create gcdOfOddEvenSums.java"*.

At the time of writing, this PR is in **open** state and waiting for review. Screenshot is shown in Figure 4.

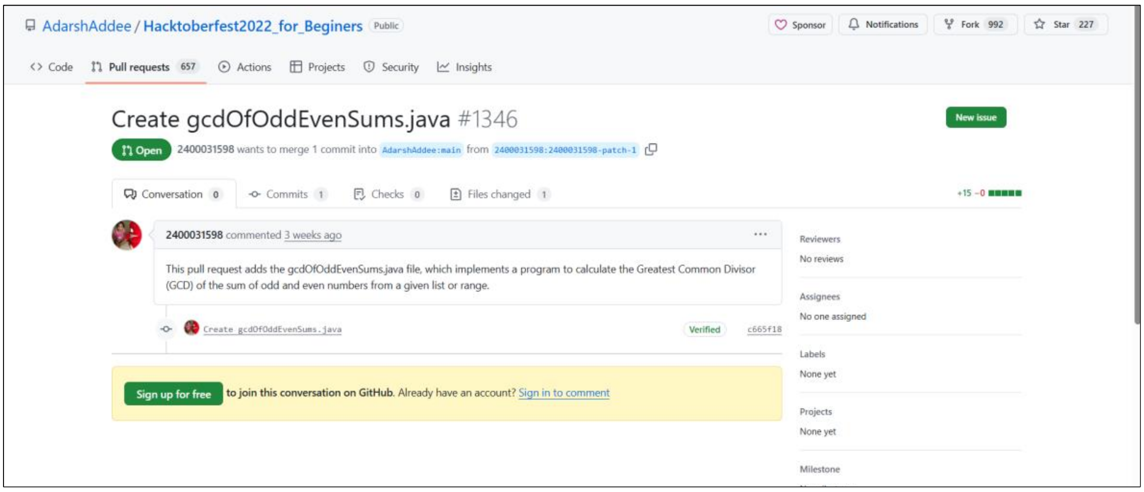


Figure 4: PR: Create gcdOfOddEvenSums.java (open)

7.4 PR 4: First Contributions – Add My Name

Repository: firstcontributions/first-contributions

Issue and Goal

This is a famous tutorial repository that teaches new developers how to make their first contribution. The goal is to add your name to the CONTRIBUTORS.md file.

What I Did

- Forked the project and cloned it locally.
- Added my name to the contributors list.
- Committed the change and raised a PR titled *“docs: add my name to CONTRIBUTORS”*.

The PR was **merged**, which completed my first official tutorial contribution. Screenshot is shown in Figure 5.

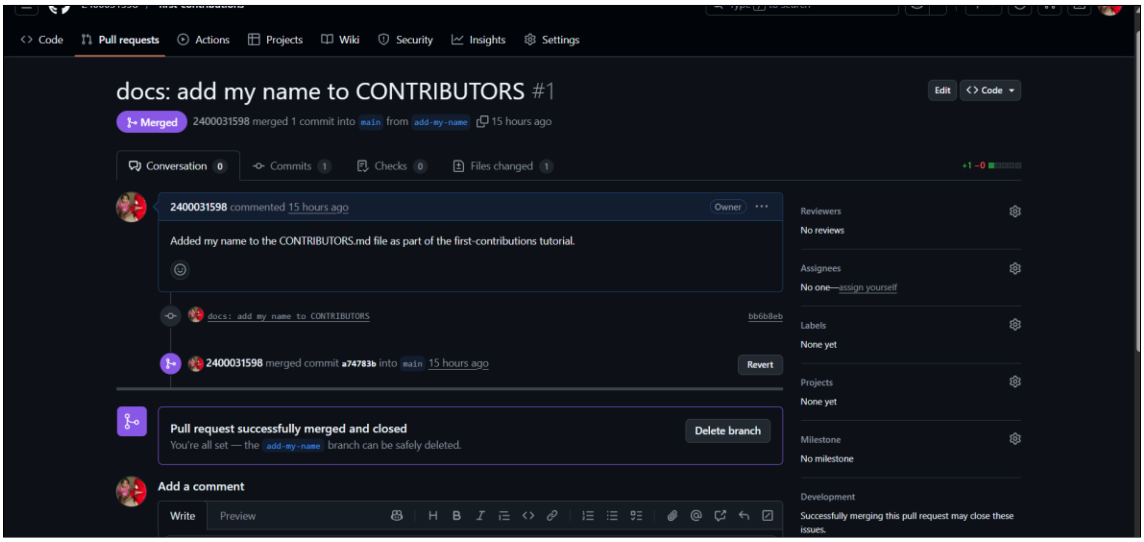


Figure 5: PR: docs: add my name to CONTRIBUTORS (merged)

8 Blog and LinkedIn Posts

8.1 Journey of Open Source (Blog Post)

I wrote a blog about my open source journey and experiences on Dev.to.

Link: https://dev.to/2400031598/my-first-open-source-journey-5epj

8.2 Self-Hosted Project – Jellyfin

I shared a LinkedIn post when my tiny Jellyfin server finally went live.

Link: https://www.linkedin.com/posts/sri-anjali-5b98ba356_my-tiny-jellyfin-server-is-finally-live-activity-7399314595494006784-tzX8?utm_source=share&utm_medium=member_android&rcm=ACoAAFjEQd0B6pFZr_TUhnlDkVcCse4EQOIpGpw

8.3 PR Merge Celebration

I also posted on LinkedIn when one of my PRs got merged.

Link: https://www.linkedin.com/posts/sri-anjali-5b98ba356_made-a-new-pr-and-got-it-merged-loving-activity-7399350416762744832-Vzg-?utm_source=share&utm_medium=member_android&rcm=ACoAAFjEQd0B6pFZr_TUhnlDkVcCse4EQOIpGpw